

SISTEMAS OPERATIVOS

Conferencia 2 · Tema II: Procesos

Carrera: Ingeniería Informática	Profesora: Ing. Claudia Vecino Fernández
Año: 3ro Período: 2do Tipo de curso: CRD	Correo: claudiavf@uho.edu.cu
Disciplina: Infraestructura de Sistemas Informáticos	Teléfono: 54196249

Objetivo: Analizar el concepto de proceso, sus estados y transiciones, la programación concurrente, los grafos de precedencia, las condiciones de concurrencia y la jerarquía entre procesos.

Sumario

- Modelo de procesos. Estados de los procesos.
- Programación concurrente. Grafos de precedencia.
- Condiciones de concurrencia. Especificaciones.
- Jerarquía entre procesos.

Bibliografía

- Tanenbaum, A. S. Sistemas Operativos Modernos. Págs. 31–37.
- Tanenbaum, A. S. Operating System: Design and Implementation. Págs. 45–51.
- Peterson, J. y Silberschatz, A. Operating System Concepts. Págs. 103–110, 307–325.

1. Modelo de Procesos. Estados de los Procesos

Como ya se indicó, el concepto de proceso juega un rol fundamental en la teoría y práctica de los sistemas operativos. Se señaló que un proceso era simplemente un programa en ejecución, lo que implicaba la tenencia de áreas de memoria, valores de registros, contador de programa, puntero de la pila, etc.

La información acerca de los procesos presentes en un instante de tiempo se conserva en la tabla de procesos, la que permite restaurar un proceso que perdió antes el control de ejecución. Cada entrada de esta tabla se corresponde con un proceso en el sistema y recibe el nombre de Bloque de Control de Proceso (PCB). Este contiene diferentes grupos de información para un proceso específico:

Campo del PCB	Descripción
Estado del proceso	Indica si el proceso está en ejecución, listo o bloqueado.

Campo del PCB	Descripción
Contador de programa	Dirección de la próxima instrucción a ejecutar.
Registros del CPU	Acumuladores, índice, propósito general, estado, etc.
Información de memoria	Dirección, inicio y longitud de segmentos, tablas de páginas, etc.
Información de contabilidad	Tiempo real y de CPU utilizado, límites de tiempo, número del proceso.
Estado de E/S	Dispositivos asignados, lista de ficheros abiertos, solicitudes de E/S.
Información de planificación	Prioridad, punteros a las colas de planificación, etc.

En el modelo de procesos todo el software ejecutable, a menudo incluyendo el propio sistema operativo, se organiza como procesos secuenciales. Aparentemente cada proceso tiene su propio procesador central, pero en realidad éste cambia de uno a otro de acuerdo con el concepto de multiprogramación (seudoparalelismo).

Los procesos son totalmente aleatorios en el tiempo y el comportamiento de un conjunto de ellos dependerá de las condiciones en un instante dado. Esto implica que los programas no pueden ser elaborados asumiendo lo que pasará en el futuro cuando se estén procesando.

1.1 Estados de los Procesos

Un proceso puede tener diferentes estados durante su existencia. El número de estados dependerá del diseño del sistema operativo, pero al menos hay tres que siempre estarán presentes:

Estado	Descripción
En ejecución	El proceso está en posesión del CPU en ese instante.
Bloqueado	El proceso está esperando hasta que ocurra un evento externo (por ejemplo, una E/S).
Listo	El proceso está en condiciones de ejecutar, pero está detenido temporalmente para permitir a otro proceso la ejecución.

1.2 Transiciones entre Estados

Si a los tres estados les agregamos las transiciones entre ellos obtenemos el grafo de estados. Las cuatro transiciones posibles son:

Transición	Descripción
1	Ejecución → Bloqueado: El proceso se bloquea porque necesita esperar la ocurrencia de un evento externo. Se logra con la ejecución de una llamada al sistema o cuando el SO recibe una solicitud que no puede ser satisfecha en ese instante.
2	Ejecución → Listo: El planificador del procesador (scheduler) decide entregar el CPU a otro proceso, aunque el actual podría continuar. Puede deberse a que el proceso ya consumió su quantum o que hay otro con mayor prioridad.
3	Listo → Ejecución: El planificador entrega el control del CPU a un proceso listo, porque el proceso anterior terminó, se bloqueó o le fue quitado el CPU.
4	Bloqueado → Listo: El evento externo que el proceso esperaba tiene lugar. Si en ese instante no hay otro proceso ejecutando o su prioridad es superior al actual, de inmediato ocurrirá también la transición 3.

✦ **Nota actualizada:** En sistemas modernos (Linux, Windows) los procesos pueden tener estados adicionales como 'Zombie' (terminó pero el padre no recogió su código de retorno) o 'Suspendido' (sacado de memoria al disco por el planificador de mediano plazo). En Linux se puede ver el estado de los procesos con el comando `ps aux` o `top`.

Haciendo uso del modelo de procesos resulta más fácil comprender cómo puede ser un sistema operativo por dentro. Algunos procesos constituyen programas mandados a ejecutar por los usuarios, pero otros son parte del sistema (sistema de ficheros, administrador de memoria y manejadores de dispositivos). Todos estos procesos se bloquean cuando necesitan esperar por algo y al ocurrir el evento pasan a listo.

La vista antes explicada permite llegar a un modelo del sistema operativo donde el nivel más bajo es el planificador y el siguiente lo forman una variedad de procesos (creados por los usuarios o partes del sistema operativo). En el planificador no solo se considera el planificador de procesos sino también los manejadores de interrupción y la comunicación entre los procesos.

2. Programación Concurrente. Grafos de Precedencia

El término programación concurrente, o simplemente concurrencia, se refiere a la ejecución paralela de instrucciones o procesos (aunque sea seudoparalela se deberán tener en cuenta las mismas consideraciones). Resulta conveniente revisar primero lo referente a las instrucciones.

Suponga que se dispone de un procesador con varias unidades funcionales para realizar las instrucciones o simplemente se tienen múltiples CPU y se desea ejecutar el siguiente grupo de instrucciones:

```
S1: a = x + y;  
S2: b = z + 1;  
S3: c = a - b;  
S4: w = c + 1;
```

Lógicamente, la instrucción S3 ($c = a - b$) no puede ser ejecutada hasta que no hayan sido realizadas S1 y S2, de forma tal que los valores de a y b hayan sido calculados. Lo mismo ocurre con S4 respecto a S3. En cambio, las instrucciones S1 y S2 se pueden ejecutar concurrentemente porque una no depende de la otra.

Lo importante de notar en este ejemplo es que dentro de un proceso existen restricciones de precedencia entre las distintas instrucciones. Para esquematizar estas situaciones se hace uso de los llamados grafos de precedencia.

Definición: Un grafo de precedencia es un grafo dirigido y sin ciclos donde los nodos corresponden a instrucciones. Un arco desde un nodo Si a un nodo Sj significa que la instrucción Sj solo puede ser ejecutada después que se realice la Si.

El grafo de precedencia para el ejemplo anterior quedaría: S1 y S2 apuntan hacia S3, y S3 apunta hacia S4. Esto indica que S1 y S2 pueden ejecutarse en paralelo, luego S3, y finalmente S4.

2.1 Instrucciones Fork / Join

Las instrucciones fork y join fueron unas de las primeras notaciones utilizadas para especificar en un programa la ejecución concurrente de instrucciones.

La instrucción fork L produce en un programa dos hilos de ejecución concurrentes: uno comienza en la etiqueta L y el otro continúa en la instrucción siguiente al fork.

La instrucción join permite unir o recombinar varios hilos de ejecución en uno solo. Dado que cada hilo debe ejecutar el join en momentos diferentes, todos terminarán menos el último. La instrucción tiene un parámetro que indica cuántos hilos se deberán unir.

```
count = 2;  
S1: fork L;  
S2: join count; goto fin;  
L: S3; join count;  
fin: S4;
```

✦ **Nota actualizada:** En la programación moderna se utilizan los hilos (threads) del sistema operativo o librerías como pthreads (POSIX), OpenMP, o las primitivas de concurrencia de lenguajes como Java (synchronized, ExecutorService) y Python (threading, asyncio). El concepto de fork/join sigue presente en los frameworks de tareas paralelas como Java ForkJoinPool.

3. Condiciones de Concurrencia. Especificaciones

Para que dos instrucciones puedan ejecutarse concurrentemente deberán cumplir ciertas condiciones, denominadas condiciones de concurrencia (condiciones de Bernstein).

Definamos el conjunto $R(S_i) = \{a_1, a_2, \dots, a_n\}$ como el conjunto de lectura para S_i , formado por todas las variables cuyos valores son referenciados en la instrucción S_i durante su ejecución.

Definamos el conjunto $W(S_i) = \{b_1, b_2, \dots, b_m\}$ como el conjunto de escritura para S_i , es decir todas las variables que cambian de valor como resultado de la ejecución de la instrucción S_i .

Condiciones de Bernstein: Para que dos instrucciones S_a y S_b puedan ser ejecutadas concurrentemente y producir un resultado correcto, se tienen que cumplir las tres condiciones siguientes: 1. $R(S_a) \cap W(S_b) = \emptyset$ 2. $R(S_b) \cap W(S_a) = \emptyset$ 3. $W(S_a) \cap W(S_b) = \emptyset$

Aplicando estas condiciones a $S_1: a = x + y$, y a $S_3: c = a - b$, veremos que efectivamente no pueden ejecutarse concurrentemente, ya que $R(S_3) \cap W(S_1) = \{a\} \neq \emptyset$, lo que viola la primera condición.

El grafo de precedencia es un medio útil para visualizar las restricciones de precedencia que puedan existir en un conjunto de instrucciones pertenecientes a un proceso, pero no permiten programar cálculos concurrentes en un lenguaje de programación. Para ello se utilizan las instrucciones fork/join u otras primitivas de sincronización.

3.1 El Problema Productor-Consumidor

Un ejemplo clásico de la problemática de la concurrencia es el problema del productor-consumidor. En él, un proceso productor genera elementos que son almacenados en un buffer compartido, y un proceso consumidor los extrae y procesa. El acceso concurrente no controlado al buffer puede producir resultados incorrectos (condición de carrera).

```
/* Ejemplo: Productor-Consumidor con buffer circular */
short int counter = 0, in = 0, out = 0;

/* Productor */
producir_elemento(&nextp);
buffer[in] = nextp;

/* Consumidor */
nextc = buffer[out];
out = (out+1) % N;
```

```
in = (in+1) % N;           counter--;
counter++;                 consumir_elemento(nextc);
```

La instrucción `counter++` implica tres operaciones en ensamblador: leer `counter`, sumarle 1, escribir el resultado. Lo mismo aplica a `counter--`. Si ambas se ejecutan de forma concurrente y no atómica, el valor final de `counter` puede ser incorrecto. Este problema motiva el estudio de los mecanismos de sincronización y exclusión mutua.

✦ **Nota actualizada:** Las condiciones de carrera (race conditions) son uno de los problemas más comunes en programación concurrente. Herramientas modernas como ThreadSanitizer (TSan) en GCC/Clang pueden detectarlas en tiempo de ejecución. En lenguajes de alto nivel, el uso de variables atómicas (`std::atomic` en C++, `AtomicInteger` en Java) resuelve el problema para operaciones simples.

4. Jerarquía entre Procesos

En los sistemas multiprogramados los procesos pueden crear nuevos procesos. La relación entre el proceso que crea y el creado se denomina relación padre-hijo. Un arco del nodo P_i al nodo P_j en el árbol de procesos significa que P_i creó a P_j . En este caso, P_i es el padre de P_j y P_j es hijo de P_i . Cada proceso tiene un solo padre, pero puede tener tantos hijos como sean necesarios.

Definición: El árbol de procesos es una estructura arborescente que refleja la jerarquía de creación entre los procesos activos en el sistema. En la raíz se encuentra el proceso inicial (`init` o `systemd` en Linux), del cual descienden todos los demás procesos.

4.1 Creación de Procesos en UNIX/Linux

En sistemas UNIX/Linux la llamada al sistema `fork()` crea un nuevo proceso (hijo) que es una copia exacta del proceso padre. Ambos procesos continúan ejecutándose desde el punto siguiente al `fork`, diferenciándose en el valor de retorno de la llamada: el padre recibe el PID del hijo y el hijo recibe 0.

```
#include <unistd.h>
pid_t pid = fork();
if (pid == 0) {
    /* código del hijo */
    execve("/bin/ls", args, env);
} else {
    /* código del padre: espera al hijo */
    wait(NULL);
}
```

Tras el `fork()`, si el hijo necesita ejecutar un programa diferente, utiliza la llamada `exec()` para reemplazar su imagen de memoria con la del nuevo programa. La combinación `fork/exec` es el mecanismo estándar de UNIX para lanzar nuevos procesos.

Llamada al sistema	Función	Ejemplo de uso
fork()	Crea un proceso hijo idéntico al padre.	Lanzar un subproceso
exec()	Reemplaza la imagen del proceso con un nuevo programa.	Ejecutar /bin/ls en el hijo
exit()	Termina el proceso y devuelve un código de salida al padre.	Finalización normal o con error
wait()	El padre se bloquea esperando la terminación de un hijo.	Sincronización padre-hijo
getpid()	Devuelve el PID del proceso actual.	Identificar al proceso
getppid()	Devuelve el PID del proceso padre.	Navegar la jerarquía

🔴 **Nota actualizada:** En Linux, el proceso inicial es systemd (PID 1), que es el padre de todos los procesos del sistema. El comando `ps tree` muestra el árbol completo de procesos. Los procesos huérfanos (cuyo padre terminó antes que ellos) son adoptados por systemd. Los procesos zombie son aquellos que terminaron pero cuyo padre no recogió su código de retorno con `wait()`.

5. Estudio Independiente

1. ¿Cuáles son los tres estados básicos de un proceso? Describa cada uno y explique las cuatro transiciones posibles entre ellos.
2. ¿Qué información contiene el Bloque de Control de Proceso (PCB)? ¿Por qué es fundamental para la multiprogramación?
3. Aplique las condiciones de Bernstein a las instrucciones S1: $x = a + b$ y S2: $y = a * c$. ¿Pueden ejecutarse concurrentemente? Justifique.
4. Explique el problema productor-consumidor. ¿Qué ocurre si `counter++` y `counter--` se ejecutan sin sincronización? Ilustre con un ejemplo de ejecución intercalada.
5. Explique la diferencia entre `fork()` y `exec()` en Linux. ¿Por qué se usan juntos habitualmente?
6. Investigue y explique el concepto de proceso zombie y proceso huérfano en Linux. ¿Cómo se evita la acumulación de procesos zombie?

— Fin de la Conferencia 2 —